

Playbook de Arquitetura: Dimensionamento de Computação (EC2)

0 Case SwiftTrack IoT: Do Modelo de Análise
à Infraestrutura Escalável.



VERSÃO: 1.0 | FASE: ELABORAÇÃO | CLOUD ARCHITECTURE

O Elo Perdido: Onde a API Django vai rodar?

Precisamos dimensionar a engrenagem principal:

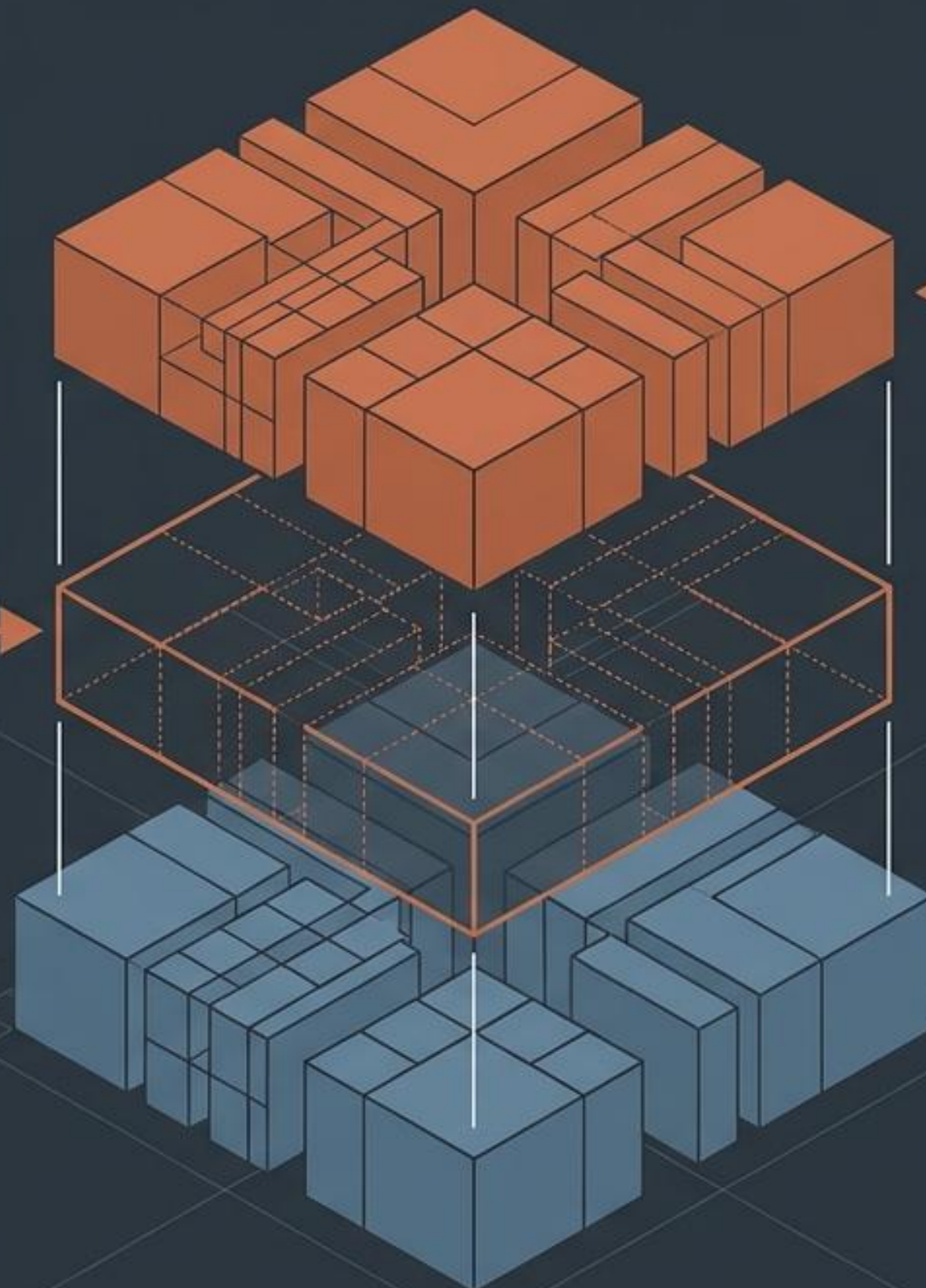
Família, quantidade de instâncias, escalabilidade e distribuição de carga.

Base Sólida (Semanas 1-4):

Rede segura configurada com VPC isolada.

Topo Sólido (Semana 5): Banco de Dados Dimensionado.

Multi-AZ, RPO 15 min,
RTO 1 hora
(Custo: ~\$173/mês).



Carga e Tráfego



1.000 TPS

Transações por Segundo no pico.
Contexto: 10.000 motoristas ativos;
500 conexões simultâneas estimadas.

Desempenho



< 200ms

Latência máxima da API.
Consumo estimado: ~0,5ms de CPU
e ~50MB de RAM por requisição.

Resiliência



99,95% SLA

Downtime máximo aceitável de
~21 minutos/mês. Picos de tráfego
de até 10x na Black Friday.

Restrição Orçamentária



US\$ 400 / mês

Teto alocado para a camada de
computação EC2 (27% do orçamento
total de US\$ 1.500).

Como traduzir Requisitos em Infraestrutura?

O Problema



Foco nos requisitos de negócio: desempenho, custo e escala.

A Tradução (Classes de Análise - RUP)



Uso de UML abstrata para definir as responsabilidades de cada componente antes de escolher o hardware. Garante rastreabilidade, facilita a comunicação e foca no O QUÊ em vez do COMO.

A Solução



Componentes AWS: Instâncias EC2, Auto Scaling e Load Balancers reais.

Matriz de Diagnóstico: Escolhendo a Família EC2

Família	Otimização	TPS Estimado	Veredito
 t3 (Burstable)	Cargas variáveis com picos.	~500 TPS (medium)	ESCOLHIDA. Melhor custo-benefício para tráfego intermitente.
 c5 (Compute-optimized)	CPU intensiva sustentada.	~1.000 TPS (large)	REJEITADA. Custo base muito alto (\$62/mês) e sem necessidade de CPU constante.
 m5 (Uso geral)	Aplicações balanceadas.	~800 TPS (large)	REJEITADA. Maior capacidade de memória do que a API Django necessita.

A Matemática da Capacidade (Por que t3.medium?)

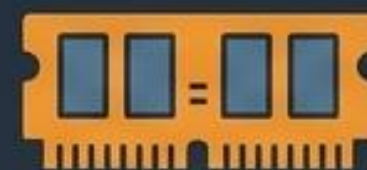
Etapa 1: Demanda de CPU



$1.000 \text{ TPS} \times 0,5\text{ms/req} = 500\text{ms/s}$ (0,5 vCPU reais).

Margem de Segurança (4x) = Necessário **2 vCPU**.

Etapa 2: Demanda de Memória



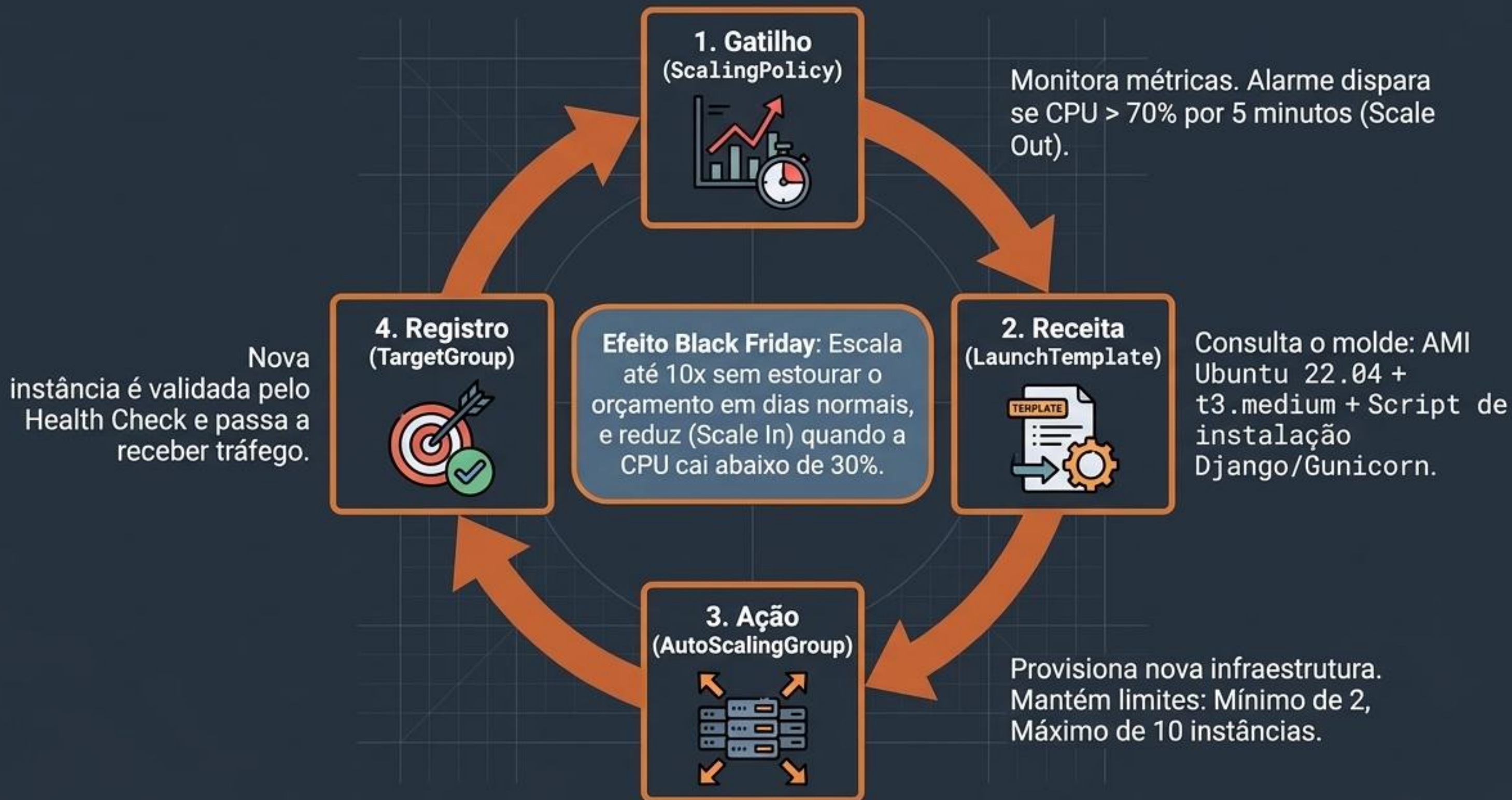
$500 \text{ conexões simultâneas} \times 50 \text{ MB/req} = 25 \text{ GB}$.

Otimização: Com uso de Connection Pooling, o consumo cai.

Necessário = **~4 GB RAM**.

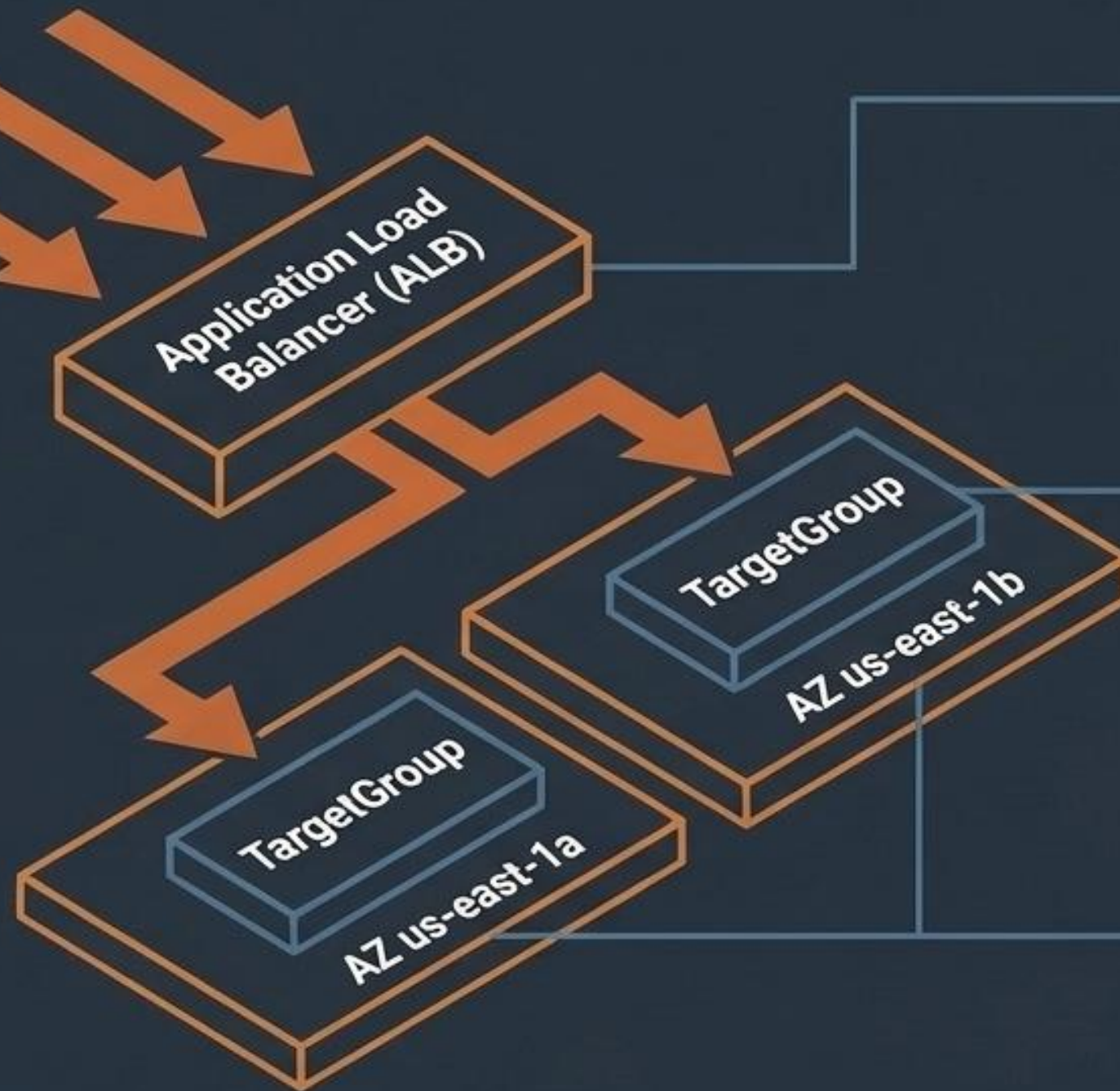
CONCLUSÃO ESTRATÉGICA: A especificação 2 vCPU e 4 GB RAM é exatamente o perfil da instância **t3.medium**. Ela suporta ~500 TPS cada. Para atender os 1.000 TPS de pico garantindo **Multi-AZ**, precisamos de **no mínimo 2 instâncias na base**.

O Motor de Elasticidade: Ciclo do Auto Scaling



Roteamento Inteligente e Alta Disponibilidade

Tráfego Internet



Application Load Balancer (ALB)

Atua na Camada 7. Gerencia terminação HTTPS (porta 443) e distribui a carga uniformemente.

Multi-AZ por Padrão

O roteamento atinge Subnet-Public-A e Subnet-Public-B simultaneamente, protegendo contra falha de um datacenter inteiro (Garante os 99,95% de SLA).

Target Group (Health Checks)

Proteção contra instâncias mortas. O ALB dispara pings para /health/ a cada 30 segundos. Se a API Django não responder 200 OK, a instância é isolada.

Síntese de Tradução: O Diagrama UML de Computação



Esta é a visão estrutural completa. Nos próximos 3 slides, faremos o raio-x detalhado de como esses blocos abstratos recebem atributos reais da AWS.

Raio-X da Classe: A Instância Base

O Modelo Abstrato

```
class ComputeInstance {  
  - instanceType: t3.medium  
  - ami: Ubuntu 22.04 LTS  
  - securityGroup: SG-EC2-API  
  - ebsVolume: 30 GB gp3  
}
```

Detalhes de Infraestrutura

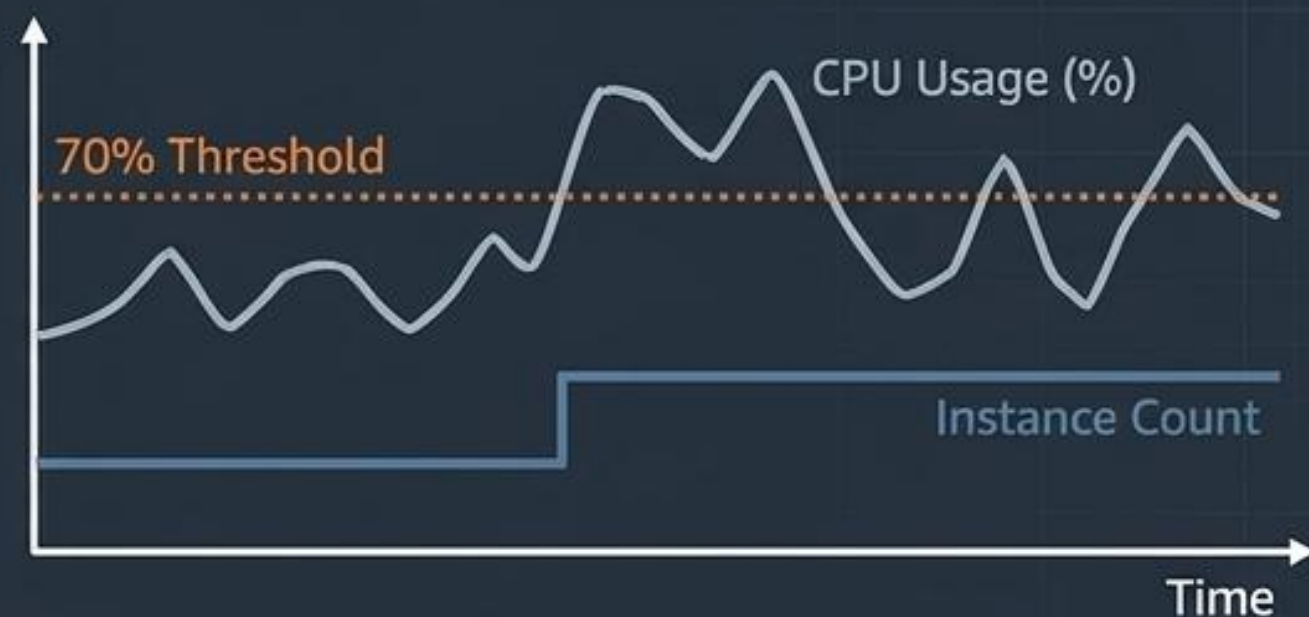
- **AMI:** Ubuntu 22.04 LTS (Escolhido pela estabilidade e suporte longo).
- **IAM Role:** EC2-API-Role (Garante acesso sem senhas fixas ao S3, Secrets e DynamoDB).
- **Storage (EBS):** 30 GB gp3 (Volume otimizado para I/O do sistema operacional).
- **User Data:** Script injetado no boot para instalar automaticamente Django, Gunicorn e Nginx.

Raio-X da Classe: Orquestração e Escalabilidade

O Modelo Abstrato

```
class AutoScalingGroup {  
  - minSize: 2  
  - maxSize: 10  
  - desiredCapacity: 2  
}  
  
class ScalingPolicy {  
  - metric: CPUUtilization  
  - threshold: 70%  
}
```

Comportamento Dinâmico



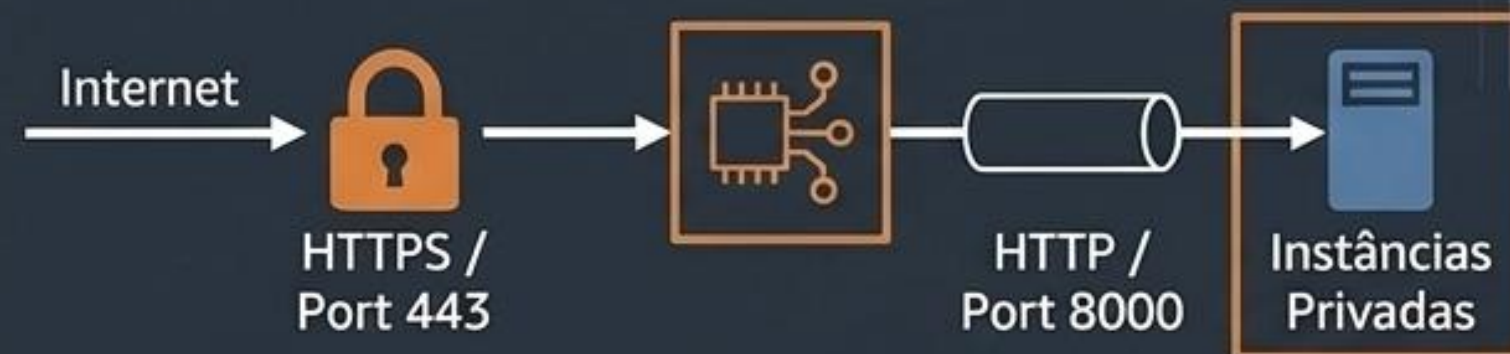
- Min/Max: Desired/Min de 2 (Garante Multi-AZ base). Max de 10 (Impede faturas surpresa).
- Grace Period: 300s. Tempo para o script de User Data rodar e o Django subir antes do Health Check falhar.
- Cooldown: 300s. Evita o “efeito sanfona” (*flapping*) em picos instáveis de tráfego.

Raio-X da Classe: Ponto de Entrada

O Modelo Abstrato

```
class TargetGroup {  
  - protocol: HTTP  
  - port: 8000  
  - healthCheckPath: /health/  
}
```

A Implementação AWS



- **Listeners do ALB:** Recebe tráfego externo via Internet na porta 443 (HTTPS).
- **Offloading:** O ALB descriptografa o tráfego e o repassa na porta 8000 (HTTP) pura para as instâncias privadas, poupando a CPU da t3.medium.
- **Stickiness:** Desativado (false). A API Django da SwiftTrack é stateless, não exige amarração de sessão, permitindo distribuição 100% fluida.

A Economia da Nuvem: Matemática Orçamentária

Teto de Orçamento (US\$ 400)

Total Base: ~\$96,00 / mês

Data Transfer Estimado (~100GB): US\$ 9,00

Application Load Balancer (ALB): US\$ 22,00

EBS (2 × 30 GB gp3): US\$ 4,80

EC2 (2 × t3.medium On-Demand):
US\$ 60,00

Análise Orçamentária:

Representa apenas ~6% do budget total do projeto e tem extrema folga em relação ao teto.

Mesmo escalando até o máximo de 10 instâncias sob pressão massiva (Black Friday), a arquitetura atingiria ~\$300/mês, respeitando com segurança a trava financeira da SwiftTrack.

Matriz de Trade-offs: As Grandes Escolhas

Decisão Arquitetural	O que Ganhamos (Prós)	O que Sacrificamos (Contras)
On-Demand vs. Reserved (Instâncias)	✓ Flexibilidade total e sem compromisso em fase de lançamento.	⚠ Deixamos de ganhar ~40% de desconto de uma Reserved Instance.
Família t3 vs. Família c5	✓ Economia de quase 50% (\$30 vs \$62) e capacidade de burst de CPU quando necessário.	⚠ Desempenho sustentado de 100% de CPU ininterrupta (não necessário para o Django).
ALB (L7) vs. NLB (L4)	✓ Roteamento inteligente, terminação HTTPS nativa e flexibilidade de Paths.	⚠ Custos marginais levemente mais altos (\$22 vs \$18) e velocidade pura L4.

Checklist de Entrega: Seu Turno (Domingo, 23:59)

[]	Diagrama PlantUML: Todas as 6 Classes com atributos e multiplicidades corretas.
[]	Memória de Cálculo: Demonstração visual de TPS -> vCPU / RAM e Instâncias.
[]	Planilha de Custos: Precificação comprovada via AWS Pricing Calculator.
[]	Matriz de Trade-offs: Documentação das renúncias (Por que não NLB? Por que não C5?).
[]	Pitch de Defesa: Sua equipe deverá convencer a banca (em 4 slides) de que os \$96/mês valem o investimento.

Next System: S3 Storage & Lifecycle Policies (Como reter dados de IoT sem falir a SwiftTrack)